DOCUMENTO DE PROYECTO · INSTANTÁNEA 29 SEP – 1 OCT 2026

BEGIZ

App personal para guardar webs de inspiración. Requisitos, plan de arquitectura y procesos, en un solo documento.

01

DOCUMENTO DE REQUISITOS · V0.1 · 29 SEP 2026

REQUISITOS

Visión general · Requisitos funcionales · Funcionalidades futuras · Modelo de datos orientativo · Dudas abiertas · Resumen de prioridades

02

PLAN DE ARQUITECTURA · V1.0 · 30 SEP 2026

PLAN DE ARQUITECTURA

Resumen · Punto de partida · Cómo se llegó a cada decisión · Arquitectura final · Seguridad · Despliegue y costes · Estado de la fase 1 · Hoja de ruta

03

DOCUMENTO DE REFERENCIA · 1 OCT 2026

CÓMO FUNCIONA BEGIZ

Piezas de la app · Iniciar sesión · Guardar una web · Imagen · Mosaico y filtros · Detalle, edición y borrado · Datos · Seguridad · Cómo trabajamos · Publicación y costes · Hoja de ruta

INSTANTÁNEA DEL INICIO DEL PROYECTO. NO SE ACTUALIZA.

ESTADO AL DÍA: ARCHITECTURE.MD

REQUISITOS

App de guardado de webs de inspiración. Estilo Moodmark, en versión sencilla.

01 VISIÓN GENERAL

La aplicación permite guardar páginas web que resultan interesantes (por su diseño, sus componentes o su estilo) para consultarlas más adelante como fuente de inspiración. Cada web guardada se representa con una captura de pantalla, su enlace, un título propio, etiquetas (tags) y notas personales.

El objetivo de la primera versión (MVP) es cubrir el ciclo básico: guardar, visualizar en mosaico, buscar y agrupar. Funcionalidades más avanzadas, como guardar componentes concretos, quedan para versiones futuras.

ALCANCE

ÁMBITO	CONTENIDO
■ DENTRO DEL MVP	Ficha de cada web, mosaico principal, búsqueda por tags y por título, colecciones.
DESEABLE EN EL MVP	Si es viable: captura de la página completa (no solo la parte visible).
FUERA DEL MVP	Futuro: guardado de componentes concretos de una web.

02 REQUISITOS FUNCIONALES

A. GUARDADO DE WEBS (FICHA DE CADA WEB)

ID	REQUISITO	DESCRIPCIÓN	PRIORIDAD
RF-01	Enlace a la página	Cada web guardada almacena la URL de la página concreta y permite abrirla desde la app.	■ IMPRESCINDIBLE
RF-02	Título personalizado	La usuaria asigna un título propio a cada web guardada (no necesariamente el título original de la página).	■ IMPRESCINDIBLE
RF-03	Captura de pantalla	Al guardar una web, la app hace automáticamente un pantallazo de la parte visible de la página, que se usa como miniatura.	■ IMPRESCINDIBLE
RF-04	Captura de página completa	Opción de capturar la página entera de arriba abajo, como la herramienta «Hacer captura de pantalla» de Firefox.	DESEABLE
RF-05	Notas	Campo de texto libre para escribir qué aspectos de la web resultan interesantes.	■ IMPRESCINDIBLE
RF-06	Tags	Añadir una o varias etiquetas a cada web para indicar las características concretas por las que se ha guardado (p. ej. «tipografía», «animaciones», «paleta de color»).	■ IMPRESCINDIBLE

NOTA DE VIABILIDAD · RF-04

Sí es posible. Hay dos caminos habituales: (1) desde un servidor, con un navegador «headless» como Playwright o Puppeteer, que tienen una opción `fullPage` para capturar la página entera; (2) desde una extensión de navegador, que captura la página tal como la ve la usuaria (incluidas sesiones iniciadas). Algunas webs con carga perezosa (lazy loading) o animaciones al hacer scroll pueden necesitar desplazarse por la página antes de capturar.

B. PÁGINA PRINCIPAL

ID	REQUISITO	DESCRIPCIÓN	PRIORIDAD
RF-07	Mosaico de webs	La página principal muestra las webs guardadas en una cuadrícula tipo mosaico, con la captura como miniatura, el título y sus tags.	■ IMPRESCINDIBLE
RF-08	Vista de detalle	Al pulsar una tarjeta del mosaico se abre la previsualización con la captura (completa si existe), enlace, notas, tags y colecciones.	■ IMPRESCINDIBLE

C. BÚSQUEDA Y FILTRADO

ID	REQUISITO	DESCRIPCIÓN	PRIORIDAD
RF-09	Buscador por tags	Filtrar el mosaico seleccionando uno o varios tags.	■ IMPRESCINDIBLE
RF-10	Buscador por título	Buscar las webs guardadas por el título que les ha puesto la usuaria.	■ IMPRESCINDIBLE

D. AGRUPACIONES (COLECCIONES)

ID	REQUISITO	DESCRIPCIÓN	PRIORIDAD
RF-11	Crear colecciones	Desde el menú de previsualización se pueden crear agrupaciones o «familias» de páginas, p. ej. «Minimalistas», «Brutalistas» o «Botones que me gustan».	■ IMPRESCINDIBLE
RF-12	Gestionar pertenencia	Añadir o quitar una web de una o varias colecciones, y ver el mosaico filtrado por colección.	■ IMPRESCINDIBLE

TAGS VS. COLECCIONES

Los **tags** describen el porqué de cada web (sus características) y sirven para buscar. Las **colecciones** son agrupaciones que crea la usuaria de forma intencionada, como carpetas o tableros temáticos. Una misma web puede tener varios tags y estar en varias colecciones.

03 FUNCIONALIDADES FUTURAS

ID	REQUISITO	DESCRIPCIÓN	PRIORIDAD
RF-F1	Guardar componentes	Guardar partes concretas de una web (un botón, una cabecera, un formulario…) y no solo la página completa, con su propia captura, tags y notas.	FUTURO

04 MODELO DE DATOS ORIENTATIVO

ENTIDAD	CAMPOS
Web	id, url, título, captura (miniatura), captura completa (opcional), notas, tags, colecciones, fecha de guardado
Tag	id, nombre
Colección	id, nombre, descripción (opcional), webs
Componente FUTURO	id, web de origen, captura del recorte, título, notas, tags

05 DUDAS ABIERTAS Y DECISIONES PENDIENTES

1. **Captura completa:** ¿se hace desde un servidor (headless) o desde una extensión del navegador?
2. **Tipo de app:** ¿aplicación web, extensión de navegador, app de escritorio o una combinación (p. ej. web + extensión para guardar)?
3. **Almacenamiento:** ¿solo en local o en la nube para acceder desde varios dispositivos?
4. **Captura fallida:** si la captura automática falla (web bloqueada, requiere login…), ¿se permite subir una captura manual?
5. **Tags:** ¿lista libre o sugerencias a partir de los tags ya usados?

Las respuestas están en la parte 02, sección 3.2.

06 RESUMEN DE PRIORIDADES

PRIORIDAD	REQUISITOS
■ **IMPRESCINDIBLE (MVP)**	RF-01 Enlace · RF-02 Título · RF-03 Captura · RF-05 Notas · RF-06 Tags · RF-07 Mosaico · RF-08 Detalle · RF-09 Buscador por tags · RF-10 Buscador por título · RF-11 Crear colecciones · RF-12 Gestionar colecciones
DESEABLE	RF-04 Captura de página completa
FUTURO	RF-F1 Guardar componentes concretos

PLAN DE ARQUITECTURA

De la concepción inicial a las decisiones tomadas.

01 RESUMEN

Este documento recoge el recorrido completo del proyecto: la idea de partida (documento de requisitos v0.1), las decisiones de arquitectura que se han ido tomando, con sus motivos y las alternativas descartadas, y el estado en el que queda la primera versión.

ASPECTO	DECISIÓN FINAL
Producto	App personal para guardar webs de inspiración: enlace, título propio, imagen, notas, tags y colecciones.
Front	Vue 3 + Vite + TypeScript, como SPA (sin renderizado en servidor).
Backend	Node + Hono + Drizzle ORM, en TypeScript.
Datos y login	Supabase: PostgreSQL + Supabase Auth (email y contraseña, registro público desactivado).
Imágenes	Disco local en desarrollo; Cloudflare R2 en producción.
Vista previa v1	Imagen subida a mano → `og:image` de la página → tarjeta con favicon y dominio. La captura automática pasa a la v2.
Diseño	Mínimo y provisional (fondo blanco, bordes negros finos, tipografía básica) hasta validar la plataforma.
Estado	Fase 1 construida. API verificada de punta a punta; pendiente de probar con el proyecto real de Supabase.

02 PUNTO DE PARTIDA: LA IDEA

La aplicación permite guardar páginas web interesantes por su diseño, sus componentes o su estilo, para consultarlas después como fuente de inspiración. Es una versión sencilla al estilo de Moodmark. Cada web guardada se representa con una imagen, su enlace, un título propio, tags y notas personales.

El objetivo del MVP es cubrir el ciclo básico: guardar, visualizar en mosaico, buscar y agrupar.

REQUISITOS Y SU ESTADO TRAS LAS DECISIONES

ID	REQUISITO	PRIORIDAD ORIGINAL	ESTADO EN EL PLAN
RF-01	Enlace a la página	■ IMPRESCINDIBLE	v1 · hecho
RF-02	Título personalizado	■ IMPRESCINDIBLE	v1 · hecho
RF-03	Captura automática de la parte visible	■ IMPRESCINDIBLE	**Movido a v2.** En v1 se sustituye por `og:image` + imagen manual
RF-04	Captura de página completa	DESEABLE	**v2 automática.** En v1 se cubre subiendo la captura a mano
RF-05	Notas	■ IMPRESCINDIBLE	v1 · hecho
RF-06	Tags	■ IMPRESCINDIBLE	v1 · hecho (con autocompletado)
RF-07	Mosaico de webs	■ IMPRESCINDIBLE	v1 · hecho
RF-08	Vista de detalle	■ IMPRESCINDIBLE	v1 · hecho (muestra la imagen completa si existe)
RF-09	Buscador por tags	■ IMPRESCINDIBLE	v1 · hecho (varios tags a la vez)
RF-10	Buscador por título	■ IMPRESCINDIBLE	v1 · hecho
RF-11	Crear colecciones	■ IMPRESCINDIBLE	v1 · hecho (desde el formulario y el detalle)
RF-12	Gestionar pertenencia a colecciones	■ IMPRESCINDIBLE	v1 · hecho (y mosaico filtrado por colección)
RF-F1	Guardar componentes concretos	FUTURO	Fase 4

03 CÓMO SE LLEGÓ A CADA DECISIÓN

Las decisiones se tomaron en este orden. Varias cambiaron por el camino al aparecer información nueva; se deja constancia de la opción inicial y del motivo del cambio.

3.1 SISTEMA DE FRONT

- **Análisis de los requisitos:** es una herramienta privada detrás de un login (nadie llega desde Google), casi toda la interacción ocurre en el cliente (mosaico, filtros combinados, detalle, colecciones) y probablemente habrá una extensión de navegador.
- **Conclusión técnica:** una SPA con Vite. Se descartan Next.js, Nuxt, Astro y SvelteKit en modo servidor porque resuelven problemas que aquí no existen (SEO, primera carga, contenido público).
- **Recomendación inicial:** React + Vite, por el tamaño de su ecosistema (masonry, recorte de imágenes para RF-F1, extensiones).
- **Decisión final: Vue 3 + Vite + TypeScript**, porque es el framework que mejor conoce la usuaria. Para una app de este tamaño, la soltura con la herramienta pesa más que la diferencia de ecosistema, y Vue cubre todo lo que pide el MVP.

3.2 BACKEND Y DUDAS ABIERTAS DEL DOCUMENTO

La usuaria pidió que el backend lo liderase un perfil de ingeniería backend. Las cinco dudas abiertas del documento de requisitos se resolvieron así:

DUDA ABIERTA	RESPUESTA	MOTIVO
¿Local o en la nube?	**Nube**	Acceso desde varios dispositivos y un único origen de datos para la web y la futura extensión.
¿Web, extensión, escritorio o combinación?	**Web ahora; extensión después**	La web da una app usable antes. La extensión se añade usando la misma API.
¿Captura desde servidor o extensión?	**Pospuesta a v2**	Inicialmente: servidor con Playwright y extensión en una segunda fase. Después se decidió aplazar la captura automática (ver 3.4).
¿Subida manual si la captura falla?	**Sí**	Siempre habrá webs que bloqueen bots o pidan login. En v1 además es la forma principal de tener una buena imagen.
¿Tags libres o sugeridos?	**Libres con autocompletado**	Flexibilidad sin duplicados: se normalizan (minúsculas, espacios) y se sugieren los ya usados.

Lenguaje: TypeScript también en el backend, para compartir tipos y validaciones con el front sin aprender otro lenguaje. **Framework:** Hono, cuyo cliente tipado permite que el front Vue conozca exactamente qué devuelve cada ruta.

3.3 LOGIN Y BASE DE DATOS

- **Propuesta inicial:** Better Auth para el login y Neon como Postgres.
- **Cambio:** la usuaria ya conocía Supabase. Se pasa a Supabase Auth y, de paso, a Supabase como base de datos: menos proveedores, y la parte delicada de seguridad (contraseñas, tokens, límites de intentos) resuelta.
- Supabase se usa solo para el login y como Postgres. **El front nunca consulta la base de datos directamente:** toda la lógica pasa por la API, que valida el token (JWT) de Supabase.
- **Método:** email y contraseña (los magic links son incómodos en una extensión). Registro público desactivado: la usuaria se crea desde el panel.
- **Imágenes fuera de Supabase Storage:** el plan gratuito da 1 GB, frente a los 10 GB sin coste de descarga de Cloudflare R2.

PEGA CONOCIDA

El plan gratuito de Supabase se pausa

Los proyectos gratuitos se pausan tras unos 7 días sin actividad. Se reactivan desde el panel sin perder datos. Si llega a molestar, las opciones son el plan Pro (25 $/mes) o migrar la base de datos, que es un Postgres estándar.

3.4 CAPTURA DE PANTALLA: APLAZADA A LA V2

La captura automática necesita un navegador sin interfaz (Playwright + Chromium) en un servidor, una cola de trabajos, control de páginas enormes, banners de cookies y memoria. La usuaria propuso dejarla para una segunda versión, y se aceptó: reduce mucho la complejidad y el riesgo de la v1.

ORDEN	FUENTE DE LA VISTA PREVIA EN V1
1	**Imagen subida por la usuaria:** pegada con Ctrl+V en cualquier parte de la página, arrastrada o elegida del disco. Permite subir capturas de página completa hechas con Firefox.
2	`og:image` **de la página:** la imagen que muestran las webs al compartir un enlace. Se copia a nuestro almacenamiento para que no se rompa.
3	**Tarjeta generada** con el favicon y el dominio. El mosaico nunca tiene huecos.

LIMITACIÓN ASUMIDA

La `og:image` suele ser una imagen de marketing, no una captura del diseño. Por eso la subida manual se ha hecho lo más cómoda posible.

Consecuencias: desaparecen Playwright, la cola de trabajos y los estados de captura, y ya no es obligatorio un contenedor con Chromium. La v2 solo tendrá que añadir un proceso que rellene los mismos campos de imagen.

3.5 DISEÑO VISUAL

Se deja para después de validar la plataforma. La v1 usa un estilo mínimo y provisional: fondo blanco, bordes negros finos y tipografía del sistema en negro.

3.6 ENTORNO DE DESARROLLO

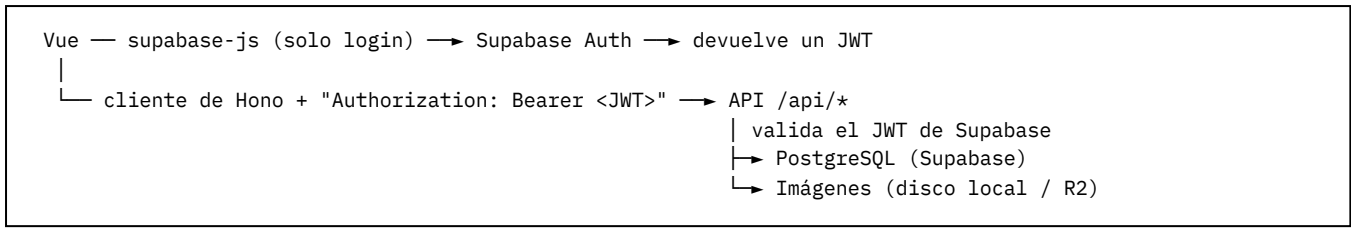
El equipo de la usuaria no tiene Docker ni la CLI de Supabase. En lugar de exigir instalarlos, se ajustó el plan: en desarrollo, la base de datos y el login van contra un proyecto de Supabase en la nube (gratis), y las imágenes se guardan en disco local. El mismo código usa R2 en producción cambiando solo la configuración.

04 ARQUITECTURA FINAL

4.1 TECNOLOGÍAS

PIEZA	ELECCIÓN	POR QUÉ
Front	Vue 3, Vite, TypeScript, Vue Router, TanStack Query	SPA ligera; caché de datos y filtros guardados en la URL.
API	Hono sobre Node	Ligero, moderno y con cliente tipado para el front.
Validación	Zod (paquete compartido)	Los mismos esquemas en front, API y futura extensión.
ORM	Drizzle	Tipado, cercano a SQL, migraciones sencillas.
Base de datos	PostgreSQL (Supabase) + `pg_trgm`	Relacional para tags y colecciones; búsqueda por título rápida.
Login	Supabase Auth	Conocido por la usuaria; la API valida el JWT.
Imágenes	Disco local / Cloudflare R2 + sharp	Conversión a WebP y miniaturas; R2 sin coste de descarga.
Repositorio	Monorepo con pnpm workspaces	Front, API y código compartido juntos y tipados entre sí.

4.2 FLUJO DE DATOS



Al guardar una web:

1. La API descarga el HTML de la página (con protección SSRF) y lee título, favicon y `og:image`.
2. Si no hay imagen propia, copia la `og:image`, la convierte a WebP y crea la miniatura.
3. Guarda la web con sus tags y colecciones en una única transacción.
4. Si la usuaria adjuntó una imagen, el front la sube después y sustituye a la anterior.

4.3 ESTRUCTURA DEL REPOSITORIO

```
webs-archive/
├── apps/
│   ├── api/          Hono + Drizzle · rutas en src/routes · migraciones en drizzle/
│   └── web/          Vue 3 + Vite · vistas en src/views · componentes en src/components
├── packages/
│   └── shared/       esquemas Zod y utilidades comunes
├── ARCHITECTURE.md   resumen técnico de este documento
└── README.md         puesta en marcha
```

4.4 MODELO DE DATOS

TABLA	CAMPOS	NOTAS
webs	id, user_id, url, title, notes, preview_key, preview_source, full_key, site_title, favicon_url, created_at, updated_at	`preview_key`: miniatura WebP del mosaico. `full_key`: imagen original subida a mano (p. ej. página completa).
tags	id, user_id, name	Nombre único por usuaria, normalizado. Los tags sin webs se borran solos.
web_tags	web_id, tag_id	Relación muchos a muchos.
collections	id, user_id, name, description	Nombre único por usuaria.
collection_webs	collection_id, web_id	Una web puede estar en varias colecciones.

`user_id` es el identificador de la usuaria en Supabase. Se preparó desde el principio para que un futuro paso a multiusuario no requiera migraciones. Al buscar por varios tags se devuelven las webs que tienen **todos** (Y), que es lo útil para buscar inspiración concreta.

4.5 API

MÉTODO	RUTA	FUNCIÓN
GET	`/api/webs?q=&tags=a,b&collection=`	Mosaico con búsqueda y filtros
POST	`/api/webs`	Guardar web (lee metadatos y `og:image`)
GET · PATCH · DELETE	`/api/webs/:id`	Detalle, edición (incluye tags y colecciones) y borrado
PUT	`/api/webs/:id/preview`	Subir imagen propia
GET · POST	`/api/collections`	Listar con contador y crear
PATCH · DELETE	`/api/collections/:id`	Renombrar y borrar (las webs se conservan)
GET	`/api/tags`	Tags en uso con contador (filtro y autocompletado)

05 SEGURIDAD

RIESGO	MEDIDA
SSRF	El servidor abre URLs que escribe la usuaria. Se bloquean las direcciones no públicas (localhost, redes privadas, metadatos del cloud), comprobándolas al conectar y en cada redirección, con tiempo y tamaño máximos.
API REST de Supabase	La clave pública del front podría leer las tablas. Se activa RLS en todas ellas sin políticas: ese acceso queda cerrado y la API propia no se ve afectada.
Acceso ajeno	Registro público desactivado. Cada consulta filtra por usuaria y no se pueden usar colecciones de otra persona.
Archivos subidos	Se valida el contenido real de la imagen (no la extensión). Tamaño máximo de 25 MB.
Imágenes públicas	Se sirven sin login, pero con rutas aleatorias imposibles de adivinar.

06 DESPLIEGUE Y COSTES PREVISTOS

PIEZA	SERVICIO	COSTE ESTIMADO
Front	Cloudflare Pages (estático)	Gratis
API	Railway, Fly.io o Render (Node). Sin Chromium en v1 caben opciones más baratas	0–5 $/mes
Base de datos + login	Supabase	Gratis (Pro: 25 $/mes si se necesita)
Imágenes	Cloudflare R2	Gratis hasta 10 GB

EN LA V2

La captura automática sí exigirá un contenedor con Chromium (unos 5 $/mes). No sirven las plataformas serverless.

07 ESTADO ACTUAL DE LA FASE 1

CONSTRUIDO

- Login, mosaico con buscador por título, filtro por varios tags y por colección.
- Guardar web con tags (autocompletado), colecciones creadas sobre la marcha e imagen por pegado, arrastre o archivo.
- Detalle con edición completa, cambio de imagen y borrado. Página de colecciones para renombrar y borrar.
- Migraciones de base de datos (extensión de búsqueda, tablas y RLS). Documentación: `ARCHITECTURE.md` y `README.md`.

VERIFICADO

- Recorrido completo de la API contra un PostgreSQL de prueba y webs reales: **34 comprobaciones superadas** (tags, filtros, colecciones, imágenes, validaciones, SSRF, borrado y aislamiento entre usuarias).
- Tipos correctos en los tres paquetes, compilación del front para producción y conexión del front con la API a través del proxy de desarrollo.

PENDIENTE DE VERIFICAR

- El front en el navegador con un login real de Supabase.
- La validación de tokens con las claves de un proyecto nuevo de Supabase (en las pruebas se usó el sistema de claves antiguo).
- El almacenamiento en R2 (solo se ha usado el disco local).

DECISIONES TÉCNICAS MENORES

- TypeScript 6 en el front: la herramienta de tipos de Vue aún no es compatible con TypeScript 7.
- Sin Pinia: los filtros viven en la URL y no hace falta otro estado global.

08 HOJA DE RUTA

FASE	CONTENIDO	ESTADO
1 · Base	Login, CRUD de webs, tags y colecciones, búsqueda, vista previa por `og:image` o imagen manual	■ **CONSTRUIDA**
Diseño	Diseño visual de la plataforma, una vez validado su funcionamiento	SIGUIENTE TRAS VALIDAR
2 · Captura	Captura automática visible y de página completa (Playwright en un worker)	PLANIFICADA
3 · Extensión	Extensión de navegador (WXT + Vue) para guardar con un clic, usando la misma API	PLANIFICADA
4 · Componentes	RF-F1: guardar partes concretas de una web con su propia imagen, tags y notas	FUTURO

PRÓXIMOS PASOS INMEDIATOS

1. Crear el proyecto de Supabase, desactivar el registro público y crear la usuaria.
2. Rellenar los archivos `.env` de la API y del front, aplicar las migraciones (`pnpm db:migrate`) y arrancar (`pnpm dev`).
3. Probar la app en el navegador con el login real.
4. Una vez validada, abrir la fase de diseño visual.

CÓMO FUNCIONA BEGIZ

Todos los procesos de la app explicados: qué haces tú, qué pasa por dentro y dónde queda guardado.

00 CONTENIDO

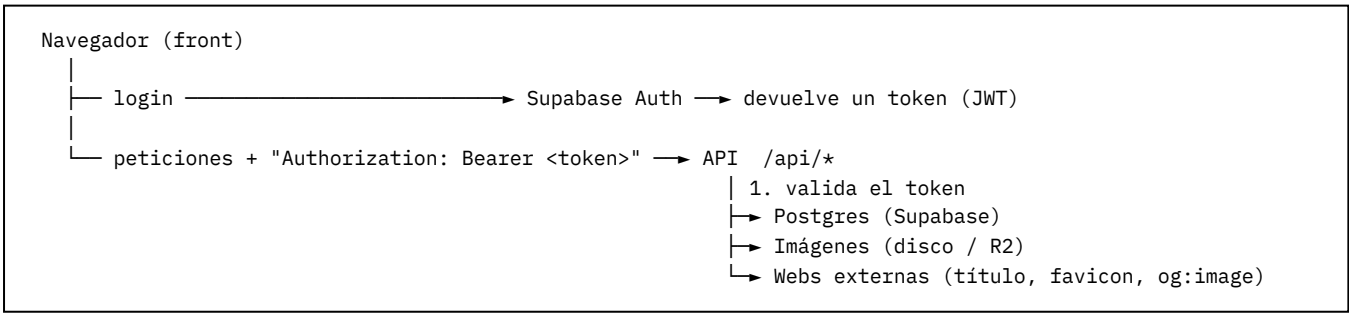
1. Las piezas de la app
2. Iniciar sesión
3. Guardar una web
4. Subir o cambiar la imagen
5. El mosaico y los filtros
6. Detalle, edición y borrado
7. Dónde se guarda cada dato
8. Seguridad
9. Cómo trabajamos
10. Publicación y costes
11. Hoja de ruta

01 LAS PIEZAS DE LA APP

Begiz se compone de cinco piezas. Tú solo ves la primera; el resto trabaja por detrás.

PIEZA	DÓNDE ESTÁ	QUÉ HACE
Front	`apps/web` · Vue 3 + Vite	La app que ves en el navegador: el mosaico, los formularios, los filtros. No guarda datos: se los pide a la API.
API	`apps/api` · Hono + Drizzle	Un programa en un servidor que recibe las peticiones del front, comprueba quién eres, lee las páginas que guardas, procesa las imágenes y habla con la base de datos.
Supabase Auth	Tu proyecto de Supabase	Gestiona las cuentas: email, contraseña cifrada y las sesiones.
Base de datos	Postgres en Supabase	Guarda las webs, tags y colecciones de cada usuaria.
Imágenes	Disco local / Cloudflare R2	Guarda los archivos de imagen. En la base de datos solo queda la ruta de cada uno.

Además hay un paquete compartido, `packages/shared`, con las reglas de validación (qué es una URL válida, cuánto puede medir un título, cómo se normaliza un tag). Lo usan el front y la API, así que las reglas son siempre las mismas en los dos lados.



LA REGLA DE ORO

El front nunca toca la base de datos

Todo pasa por la API, que es la única que sabe las reglas. Así la futura extensión del navegador podrá usar la misma API sin repetir nada.

02 INICIAR SESIÓN

TÚ
Escribes email y contraseña en `/login`.

→

SUPABASE AUTH
Comprueba la contraseña y devuelve un token firmado (JWT).

→

NAVEGADOR
Guarda la sesión para que no tengas que volver a entrar.

→

API
En cada petición valida el token y saca tu identificador.

- **El token** es como un pase firmado por Supabase que dice «esta persona es la usuaria X» y caduca pasado un tiempo. La API comprueba la firma con las claves públicas de tu proyecto (`apps/api/src/auth.ts`), así que nadie puede fabricar uno falso.
- Si el token falta, es falso o ha caducado, la API responde **401** y el front te devuelve al login.
- El **registro público está desactivado**: las cuentas solo se crean desde el panel de Supabase.
- El router del front (`apps/web/src/router.ts`) también te manda al login si intentas abrir una página sin sesión, y al volver te lleva a donde ibas.

03 GUARDAR UNA WEB

Es el proceso más completo de la app. Empieza en `/new` (`NewWebView.vue`) y lo resuelve la ruta `POST /api/webs` (`apps/api/src/routes/webs.ts`).

#	QUIÉN	QUÉ PASA
1	**Tú**	Rellenas enlace, título, notas, tags y colecciones. Opcionalmente pegas, arrastras o eliges una imagen.
2	**Front**	Envía los datos a la API. Si has puesto imagen propia, le dice que *no* descargue la de la página (`usePageImage: false`).
3	**API**	**Valida** los datos con las reglas de `packages/shared`: URL http/https, título no vacío (máx. 200), notas (máx. 5000), hasta 30 tags de 40 caracteres. Los tags se normalizan: « Dark Mode » pasa a «dark mode», y los repetidos se quitan.
4	**API**	**Lee la página** con protección SSRF (ver sección 8): título (`og:title` o `<title>`), favicon y `og:image`. Si la página no responde, no pasa nada: la web se guarda igual, sin esos datos.
5	**API**	Si toca usar la imagen de la página, la **descarga y crea la miniatura** WebP (`createThumbnail`, en `lib/preview.ts`) y la guarda en el almacenamiento. Si falla, la web queda sin imagen.
6	**Base de datos**	En **una sola transacción** guarda la web, crea los tags que no existían, los enlaza y añade la web a sus colecciones. Si algo falla (p. ej. una colección que no es tuya), no se guarda nada y se borra la miniatura que se acababa de crear.
7	**Front**	Si elegiste imagen propia, ahora la sube (ver sección 4). Después refresca el mosaico, los tags y los contadores, y te lleva al detalle de la web.

CONCEPTO

¿Qué es una transacción?

Un grupo de operaciones en la base de datos que se hacen todas o ninguna. Así nunca queda una web a medias, por ejemplo guardada pero sin sus tags.

PRIORIDAD DE LA IMAGEN DE VISTA PREVIA

ORDEN	FUENTE
1	**Tu imagen**, subida a mano. Puede ser una captura de página completa.
2	La `og:image` de la página: la imagen que muestran las webs al compartir un enlace. Se copia a nuestro almacenamiento para que no se rompa si la web la cambia.
3	Una **tarjeta generada** con el favicon y el dominio (`WebThumb.vue`). El mosaico nunca tiene huecos.

04 SUBIR O CAMBIAR LA IMAGEN

Ocurre al guardar una web con imagen propia o con «Change image» en el detalle. Ruta: `PUT /api/webs/:id/preview`.

1. **Tú** pegas una imagen con Ctrl+V (funciona en cualquier parte de la página), la arrastras o la eliges del disco (`ImagePicker.vue`).
2. La **API comprueba que la web es tuya**; si no, responde 404.
3. **Valida el contenido real** del archivo con la librería `sharp` (`detectUploadFormat`). No se fía de la extensión ni de lo que dice el navegador. Acepta PNG, JPG, WebP, GIF y AVIF, hasta 25 MB.
4. **Crea la miniatura**: WebP de 800 px de ancho. Si la imagen es muy alta (una captura de página completa), la miniatura se recorta a los 1200 px de arriba.
5. **Guarda dos archivos**: la miniatura (para el mosaico) y el original (para el detalle, a tamaño completo).
6. Actualiza la web en la base de datos y **borra las imágenes anteriores**, para no acumular archivos huérfanos.

Cada archivo lleva un identificador aleatorio nuevo en su nombre, así que una imagen nunca se sobrescribe: se crea otra y se borra la vieja. Esto permite que el navegador las guarde en caché para siempre.

05 EL MOSAICO Y LOS FILTROS

La página principal (`HomeView.vue`) pide las webs a `GET /api/webs` con los filtros activos.

FILTRO	EN LA URL	CÓMO FUNCIONA
Buscar por título	`?search=portfolio`	Busca el texto dentro del título, sin distinguir mayúsculas. Los caracteres especiales (`%`, `_`) se tratan como texto normal. Un índice especial (`pg_trgm`) mantiene la búsqueda rápida aunque haya miles de webs.
Tags	`?tags=dark,serif`	Devuelve solo las webs que tienen **todos** los tags elegidos (Y), que es lo útil para buscar inspiración concreta.
Colección	`?collection=<id>`	Solo las webs de esa colección. Se combina con los demás filtros.

- **Los filtros viven en la URL**: si recargas, compartes el enlace o vas hacia atrás, se mantienen.
- El buscador espera **250 ms** después de la última tecla antes de pedir resultados, para no lanzar una petición por cada letra.
- Las webs salen **de la más nueva a la más antigua**.

- **Caché (TanStack Query)**: el front recuerda las respuestas de la API. Al volver a una página ya vista, la muestra al instante. Al cambiar un filtro, mantiene el resultado anterior en pantalla hasta que llega el nuevo, sin parpadeos. Después de guardar, editar o borrar, invalida esa caché para que todo se vuelva a pedir actualizado.
- La lista de tags del filtro (`GET /api/tags`) muestra solo los que están en uso, **ordenados por cuántas webs los usan**. La misma lista sirve para el autocompletado al escribir tags.

06 DETALLE, EDICIÓN Y BORRADO

VER Y EDITAR UNA WEB

- El detalle (`/webs/:id`) muestra la imagen a tamaño completo si existe, el enlace a la web, el título original de la página y el formulario de edición.
- Al guardar cambios (`PATCH /api/webs/:id`) solo se actualiza lo que envías. Los **tags se reemplazan** por la lista nueva, y los que ya no usa ninguna web **se borran solos**. Las **colecciones también se reemplazan**, comprobando que todas son tuyas.

QUÉ SE BORRA Y QUÉ SE CONSERVA

ACCIÓN	SE BORRA	SE CONSERVA
Borrar una web	La web, sus enlaces con tags y colecciones, sus imágenes y los tags que quedan sin uso.	Las colecciones (con una web menos) y los tags que usan otras webs.
Borrar una colección	La colección y la lista de sus webs.	**Todas las webs**. Solo dejan de pertenecer a esa colección.
Quitar un tag de una web	El enlace; y el tag, si ya no lo usa ninguna web.	La web y el resto de sus tags.

COLECCIONES

- Se crean desde el formulario de guardar o editar una web (`CollectionPicker.vue`).
- En `/collections` se listan con su número de webs y se pueden renombrar o borrar.
- No puede haber dos colecciones con el mismo nombre: la API responde **409** con un mensaje que se muestra en pantalla.

07 DÓNDE SE GUARDA CADA DATO

DATO	DÓNDE	DETALLES
Cuenta	Supabase Auth Panel → Authentication → Users	Email y contraseña cifrada. Cada usuaria tiene un identificador único (uuid).
Webs	Tabla `webs`	Enlace, título, notas, título y favicon de la página, rutas de la miniatura y del original, origen de la imagen y fechas.
Tags	Tablas `tags` y `web_tags`	Los nombres de los tags y qué tags tiene cada web. Un nombre no se repite dentro de la misma usuaria.
Colecciones	Tablas `collections` y `collection_webs`	Nombre y descripción de cada colección, y qué webs contiene.
Imágenes	Desarrollo: `apps/api/.data/uploads/` Producción: Cloudflare R2	Organizadas como `<usuaria>/<web>/<id>.webp`. La base de datos solo guarda la ruta.
Sesión	Tu navegador	Solo el token de login. Ningún dato de la app se guarda en el navegador.

Todas las usuarias comparten las mismas tablas. Cada fila lleva una columna `user_id` con su dueña, y la API filtra por ella en **todas** las consultas.

> OJO
>
> **Las imágenes en local solo están en tu ordenador**
>
> Mientras se trabaja en local, las imágenes solo existen en tu ordenador, y la carpeta `.data` no se sube a GitHub. Si se borra o cambias de equipo, las webs siguen en Supabase pero salen con la tarjeta de favicon. Al publicar, estas imágenes no pasan solas a R2: habrá que copiarlas si se quieren conservar.

08 SEGURIDAD

RIESGO	CÓMO SE EVITA
Ver datos de otra persona	Cada consulta filtra por tu `user_id`. Si pides una web o colección que no es tuya, la API responde 404 como si no existiera, y no puedes meter tus webs en colecciones ajenas. Los tests de aislamiento lo comprueban en cada cambio.
Acceso directo a la base de datos	Supabase permite leer tablas con la clave pública que usa el front. Por eso todas las tablas tienen **RLS** (Row Level Security) activado y sin permisos: ese acceso queda cerrado. La API se conecta como propietaria y no le afecta.
SSRF	La API abre las URLs que escribes. Alguien podría usarla para alcanzar direcciones internas del servidor (`localhost`, redes privadas, metadatos del proveedor cloud). `lib/safe-fetch.ts` bloquea las direcciones no públicas **en el momento de conectar** y en cada redirección, con un tiempo máximo de 8 s y un tamaño máximo de descarga.
Archivos maliciosos	Las imágenes se validan por su contenido real, no por la extensión. Un archivo que no es imagen se rechaza aunque se llame `.png`.
Imágenes públicas	Se sirven sin login, pero sus rutas llevan identificadores aleatorios imposibles de adivinar.
Cuentas ajenas	Registro público desactivado. Solo existen las cuentas creadas desde el panel.

09 CÓMO TRABAJAMOS

COMANDOS

COMANDO	PARA QUÉ
`pnpm dev`	Arranca el front (`localhost:5173`) y la API (`localhost:8787`). Los cambios en el código se recargan solos.
`pnpm test`	Ejecuta los tests de la API y de `shared`. Unos 25 segundos, sin Supabase ni internet.
`pnpm typecheck`	Comprueba los tipos de TypeScript en todo el proyecto.
`pnpm build`	Genera la versión de producción del front.
`pnpm db:generate`	Crea una migración después de cambiar el esquema (`apps/api/src/db/schema.ts`).
`pnpm db:migrate`	Aplica las migraciones pendientes a la base de datos de Supabase.

LOS TESTS

- Usan **PGlite**, un Postgres de verdad que se ejecuta en memoria y se crea con las mismas migraciones. Cada test empieza con la base de datos vacía y nunca toca tu Supabase.
- Firman tokens igual que Supabase, así que **pasan por la validación real del login**.
- Las páginas externas se **simulan** (`test/fake-web.ts`): los tests no dependen de que una web real esté disponible.

- Cubren login, guardado, filtros, edición, borrado, imágenes, colecciones, aislamiento entre usuarias y SSRF: 53 tests en la API y 11 en `shared`.

CAMBIAR LA BASE DE DATOS

1. Se edita el esquema en `apps/api/src/db/schema.ts`.
2. `pnpm db:generate` crea un archivo SQL con el cambio en `apps/api/drizzle/`. Toda tabla nueva lleva RLS activado.
3. `pnpm test`: los tests crean su base de datos con esa migración, así que un error sale aquí antes de tocar Supabase.
4. `pnpm db:migrate` la aplica en Supabase. Las migraciones aplicadas ya no se editan nunca.

ANTES DE DAR ALGO POR TERMINADO

Verificación mínima

`pnpm test` y `pnpm typecheck` en verde (y `pnpm build` si se toca el front). Cada cambio en la API lleva sus tests. Los commits se hacen cuando tú lo pides, con mensajes en inglés.

---

10 PUBLICACIÓN Y COSTES

PIEZA	SERVICIO	GRATIS HASTA	CUÁNDO SE PAGA
Front	Cloudflare Pages (estático)	Prácticamente sin límite a esta escala	Nunca, para este uso
API	Railway, Fly.io o Render (Node)	Sin plan gratuito útil (el de Render se duerme y tarda 30–60 s en despertar)	~0–5 $/mes desde el principio
Base de datos + login	Supabase	500 MB (cientos de miles de webs)	Pro, 25 $/mes, solo si hiciera falta
Imágenes	Cloudflare R2	10 GB, descargas siempre gratis	~0,015 $ por GB y mes por encima de 10 GB

- **El límite real son las imágenes.** Con miniaturas de `og:image` (50–150 KB) caben unas 100.000 webs en 10 GB. Con capturas de página completa subidas a mano (2–5 MB cada una), unas 2.000–5.000.
- **Supabase gratuito se pausa** tras 7 días sin uso. Se reactiva desde el panel sin perder datos.
- **Sin copias de seguridad descargables** en el plan gratuito: al publicar, conviene preparar una exportación periódica.

Cifras del plan de arquitectura (septiembre de 2026). Los proveedores cambian sus planes; se revisarán antes de publicar.

11 HOJA DE RUTA

FASE	CONTENIDO	ESTADO
1 · Base	Login, webs, tags, colecciones, búsqueda, vista previa por `og:image` o imagen manual	■ VALIDADA
1b · Tests	Vitest, tests de la API y de `shared`; código revisado (inglés, nombres, comentarios)	■ HECHA
Diseño	Diseño visual de la plataforma	SIGUIENTE
2 · Captura	Captura automática con Playwright (necesita un servidor con Chromium, ~5 $/mes)	PLANIFICADA
3 · Extensión	Extensión del navegador para guardar con un clic, usando la misma API	PLANIFICADA
4 · Componentes	Guardar partes concretas de una web con su imagen, tags y notas	FUTURO
Idiomas	Selector de idioma: inglés (por defecto) y español	FUTURO
Tema	Cambio entre tema claro y oscuro	FUTURO

Para el detalle técnico de cada decisión: `ARCHITECTURE.md`. Para poner la app en marcha: `README.md`.